



Minicurso de C

Aula 4 - Estruturas de armazenamento

1

Arrays



O que são arrays?

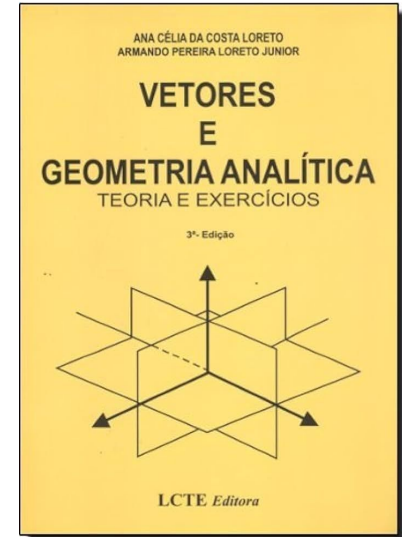
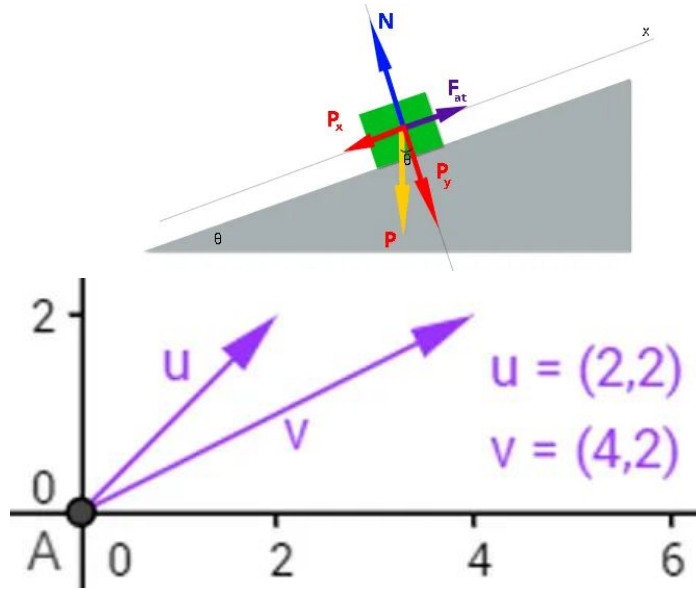
- Arrays armazenam múltiplos valores do mesmo **tipo**;
- Podem ser unidimensionais (**vetores**) ou multidimensionais (**matrizes**);
- **Strings** são arrays de caracteres.

2

Vetores



Hoje não falaremos desses vetores, Sir. Isaac Newton !!!





Declaração de um vetor

<tipo> <nome>[<tamanho>;

int numeros[5];



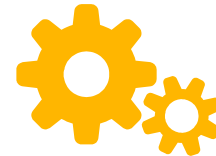
Formas de atribuição

Durante a declaração

```
int numeros[5] = {1,2,3,4,5};
```

Após a declaração

```
numeros[<posição>] = {<valor>;
```



Exemplo prático

[ACESSE AQUI](#)



Percorrendo um vetor

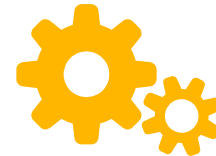
```
int notas[15];  
for( int i = 0; i < 15; i++)  
{  
    printf("%d", notas[i]);  
}
```

Posição

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----

Valores

4	8	8	6	9	9	9	2	3	2	1	7	7	5	5
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



Exemplo prático

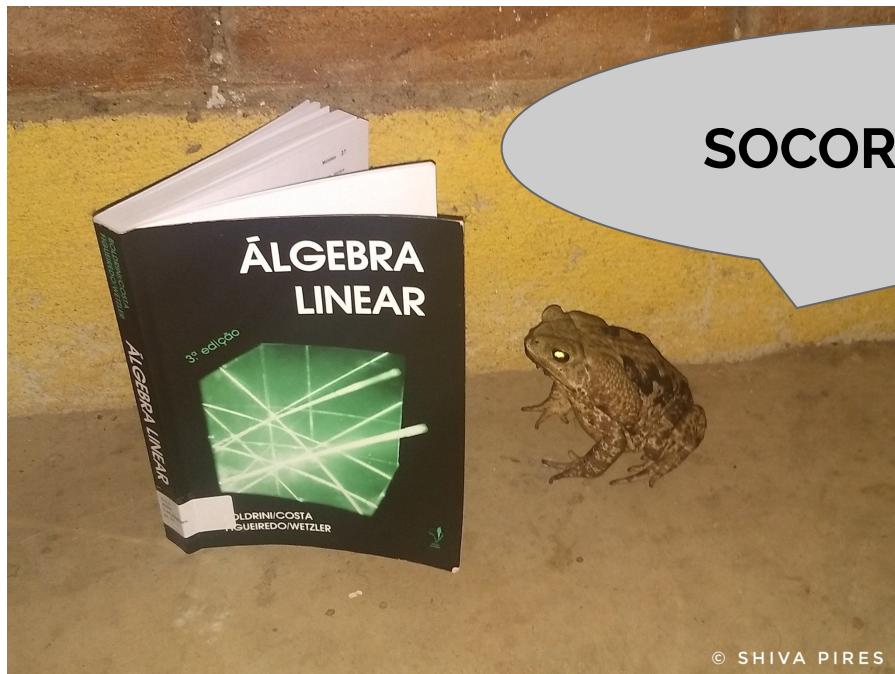
[ACESSE AQUI](#)

3

Matrices



Agora todos abram o Boldrini !!!



SOCORROOOOOOOO !!!!

$$k \cdot \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} = \begin{bmatrix} k \cdot a_{11} & k \cdot a_{12} & \cdots & k \cdot a_{1n} \\ k \cdot a_{21} & k \cdot a_{22} & \cdots & k \cdot a_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ k \cdot a_{m1} & k \cdot a_{m2} & \cdots & k \cdot a_{mn} \end{bmatrix}$$

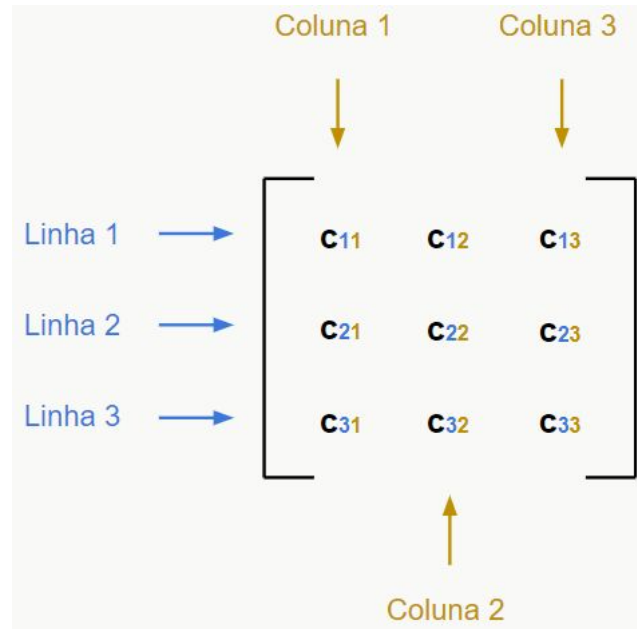
Exemplo:

$$2 \cdot \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} = \begin{bmatrix} 2 \cdot 1 & 2 \cdot 2 & 2 \cdot 3 \\ 2 \cdot 4 & 2 \cdot 5 & 2 \cdot 6 \end{bmatrix} = \begin{bmatrix} 2 & 4 & 6 \\ 8 & 10 & 12 \end{bmatrix}$$



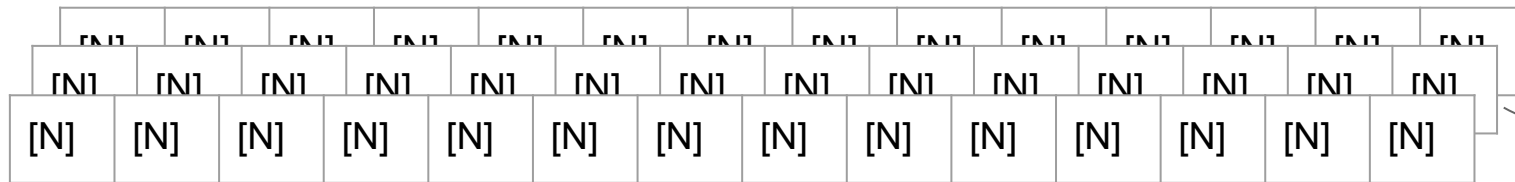
O que é uma matriz?

Enquanto um vetor armazena uma sequência de elementos em uma única linha (ou coluna), uma **matriz** permite armazenar dados em **múltiplas linhas e colunas**, formando uma **tabela de valores**.

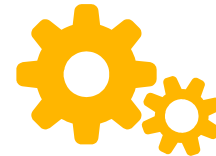




Estrutura multidimensional de uma matriz



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----



Exemplo prático

[ACESSE AQUI](#)



Percorrendo uma matriz

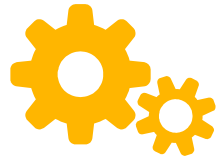
```
#include <stdio.h>

int main() {
    // Declaração e inicialização de uma matriz 3x3
    int matriz[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    // Exibindo a matriz usando dois loops aninhados
    printf("Matriz percorrida linha por linha:\n");

    for (int i = 0; i < 3; i++) { // Percorre as linhas
        for (int j = 0; j < 3; j++) { // Percorre as colunas
            printf("%d ", matriz[i][j]); // Exibe o elemento atual
        }
        printf("\n"); // Quebra de linha ao final de cada linha da matriz
    }
}
```

- O primeiro **for** percorre as linhas (índice **i**).
- O segundo **for** percorre as colunas (índice **j**).
- **printf("%d ", matriz[i][j]);** exibe cada elemento.



Exercício

Crie um programa em C que:

1. Leia duas matrizes **A** e **B** de ordem **3x3** preenchidas pelo usuário.
2. Calcule a soma das duas matrizes e armazene o resultado em uma terceira matriz **C**.
3. Imprima as matrizes **A**, **B** e **C** formatadas na tela.

Strings



Strings em C

Strings em C são representadas como arrays de caracteres terminados pelo caracter nulo (`\0`). Diferente de outras linguagens, não existe um tipo específico string, então usamos **char arrays**.



Declarando Strings

```
char palavra1[] = "UFSC"; // Forma automática (já inclui '\0')  
char palavra2[10] = "Engenharia"; // Deve ter espaço suficiente  
char palavra3[] = {'C', 'o', 'd', 'e', '\0'}; // Forma manual
```



Entrada e saída de strings

```
char nome[50];

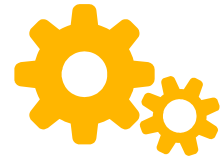
printf("Digite seu nome: ");
scanf("%s", nome); // Lê até o primeiro espaço (não recomendado para frases)

printf("Olá, %s!\n", nome);
```



Entrada e saída de strings

```
1 char nome[50];  
2  
3 printf("Digite seu nome: ");  
4 //scanf("%s", nome); // Lê até o primeiro espaço (não recomendado para frases)  
5 fgets(nome, 50, stdin); //Lê a oração completa  
6 printf("Olá, %s!\n", nome);  
7
```



Exemplo prático

[ACESSE AQUI](#)

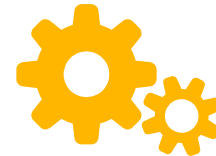
Structs



O que são structs?

As **structs** ou **estruturas** permitem criar tipos personalizados que **agrupam múltiplas variáveis** .

```
Livro titulo : Título genérico  
Livro autor : Blog Trybe  
Livro assunto : Um livro bem genérico  
Livro id_livro : 6495407  
Livro titulo : Outro título genérico  
Livro autor : Blog Trybe 2  
Livro assunto : Mais um livro bem genérico  
Livro id_livro : 6495700
```



Exemplo prático

[ACESSE AQUI](#)

9

Union

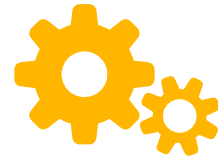


O que são uniões?

As uniões (**union**) são semelhantes a struct, mas todas as variáveis compartilham o mesmo espaço de memória.

Apenas um membro pode ser usado por vez.





Exemplo prático

[ACESSE AQUI](#)

10

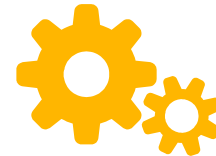
Enum



O que são enumerações?

As enumerações (**enum**)
definem um **conjunto de
valores inteiros
nomeados**, melhorando a
legibilidade do código.





Exemplo prático

[ACESSE AQUI](#)



Obrigado!

Alguma dúvida?